

Shader 优化

摘要

本文档介绍 Shader 优化的相关技术和实践方法。

1 分支语句性能优化

1.1 分支语句（if/else/switch）造成性能下降的原因

GPU 使用 SIMT（Single Instruction, Multiple Threads，单指令多线程）模型：

- 一个 warp（NVIDIA，32 线程）或 wavefront（AMD，64 线程）必须执行
- 如果部分线程走 if，部分走 else 会导致 GPU 串行执行两个分支，其它线程停顿（stall）
- 最坏情况下，性能下降 50% 甚至 90%+

注意: 如果整个 warp 的线程都走同一个分支，则不会有性能损失。

1.2 优化策略

1.2.1 避免“每像素/每顶点”随机分支

```
1  // fragment shader
2  if (textureColor.r > 0.5) {
3      color = vec3(1.0, 0.0, 0.0);
4  } else {
5      color = vec3(0.0, 0.0, 1.0);
6  }
```

对于上面这样的情况，由于向量像素颜色可能不同，导致 warp 内线程走不同分支，产生严重的性能损失。

可进行如下优化：

```
1 float threshold = step(0.5, textureColor.r); // return 0.0 or 1.0
2 color = mix(vec3(0.0, 0.0, 1.0), vec3(1.0, 0.0, 0.0), threshold);
```

1.2.2 将分支移到“warp”外或“uniform 条件”

如果条件对整个 draw call 或整个 warp 相同，编译器会自动优化，示例如下：

```
1 // uniform 条件
2 if (u_EnableBloom) {
3     color = applyBloom(color);
4 }
```

编译器会生成两个 shader variant，运行时选择其一，无运行时分支。

1.2.3 用 LUT 或纹理代替复杂 switch

```
1 switch (materialType) {
2     case 0: return metalBDRF(...);
3     case 1: return dielectricBDRF(...);
4     case 2: return clothBDRF(...);
5 }
```

在上面的例子中，如果不同像素对应的材质不同，会造成严重的分支发散。

优化策略：

- 预计算 BRDF 到纹理（如 LUT 应用于 GGX）
- 用统一公式 + 参数控制

1.2.4 用位运算或数学表达

```
1 if (a) {
2     x = 1;
3 } else if (b) {
```

```

4         x = 2;
5     } else {
6         x = 3;
7     }
8
9     // 方法1: 用逻辑运算
10    x = a ? 1 : (b ? 2 : 3);
11
12    // 方法2: 用数学表达
13    x = float(a) * 1.0 + float(!a && b) * 2.0 + float(!a && !b) * 3.0;

```

1.2.5 避免动态循环 + 分支组合

```

1    for (int i = 0; i < numLights; i++) {
2        if (lights[i].type == POINT) {
3            // point light shading
4        } else if (lights[i].type == DIRECTIONAL) {
5            // directional light shading
6        }
7    }

```

如果 numLights 是 uniform 的, 且 lights[i].type 在 warp 内相同, 则不会有问題; 否则会有严重性能损失。

优化策略:

- 将光源按类型分类 (point/spot/directional 分开处理)
- 使用 clustered shading / tiled deferred shading 等技术减少 per-pixel 光源数量

2 循环语句性能优化

2.1 循环导致性能下降的原因

2.1.1 静态循环和动态循环

类型	示例	GPU 行为	性能
静态循环 (Static Loop)	for (int i = 0; i < 8; i++)	编译器自动展开 (unroll)	极快 (无分支开销)
动态循环 (Dynamic Loop)	for (int i = 0; i < lightCount; i++)	无法展开, 运行时分支 + 计数	慢 (有分支开销)

2.1.2 循环发散 (Loop Divergence)

- 同一 warp 中，不同线程的循环次数不同，导致 GPU 必须执行最大迭代次数，短循环线程在后期空转 (stall)
- 最坏情况下，1 个线程循环 100 次，其余 31 个线程只循环一次，这 31 个线程浪费 99 次迭代时间

2.2 优化策略

2.2.1 尽量使用静态循环 (编译期已知次数)

```
1 // 静态循环，编译器展开，无循环指令
2 for (int i = 0; i < 4; i++) {
3     color += texelFetch(u_Textures[i], texCoord, 0);
4 }
```

2.2.2 强制展开小循环

HLSL(DirectX):

```
1 #pragma unroll
2 for (int i = 0; i < 8; i++) {
3     // ...
4 }
```

GLSL(OpenGL/Vulkan):

```
1 for (int i = 0; i < 8; i++) {
2     // 多数驱动会自动展开小循环
3 }
```

注意: 不要对大循环 (>16) 使用强制展开，会导致:

- 指令缓存溢出 (Instruction Cache Overflow)
- 寄存器压力过大，导致 occupancy 下降

2.2.3 将动态循环转为静态上限 + 早期退出

```

1  const int MAX_LIGHTS = 8;
2  uniform int u_ActualLightCount;
3
4  for (int i = 0; i < MAX_LIGHTS; i++) {
5      if (i >= u_ActualLightCount) break; // 早期退出
6      // 处理光源
7  }

```

- 所有线程循环次数相同（由 *u_ActualLightCount* 决定）
- *if (i >= ...)* 是 uniform 分支，无 warp 发散

2.2.4 减少循环体内昂贵操作

- 避免循环内采样纹理（尤其是不同坐标），这会破坏纹理缓存
- 避免循环内写 UAV，这会导致原子竞争
- 避免循环内复杂分支

2.2.5 分层/分块处理

对于需要处理大量数据的场景（如 100 个光源），不要再 shader 中进行循环，可以考虑：

- Clustered Shading：将屏幕分块，每块只处理相关光源
- Tiled Deferred：每个 tile 独立处理光源列表
- 多 Pass：每个 pass 处理固定数量的光源